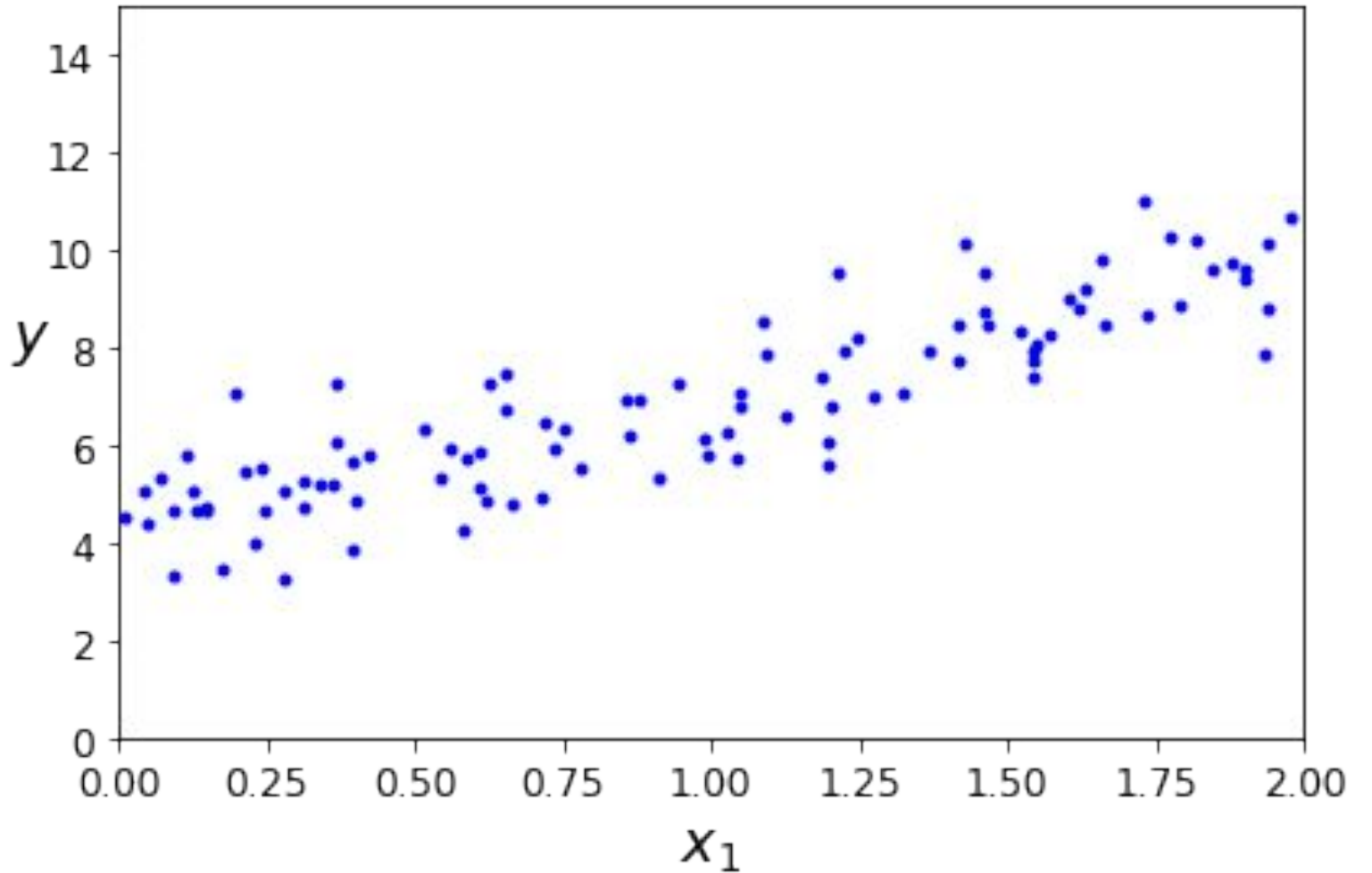


Regression

https://github.com/ageron/handson-ml2/blob/master/04_training_linear_models.ipynb

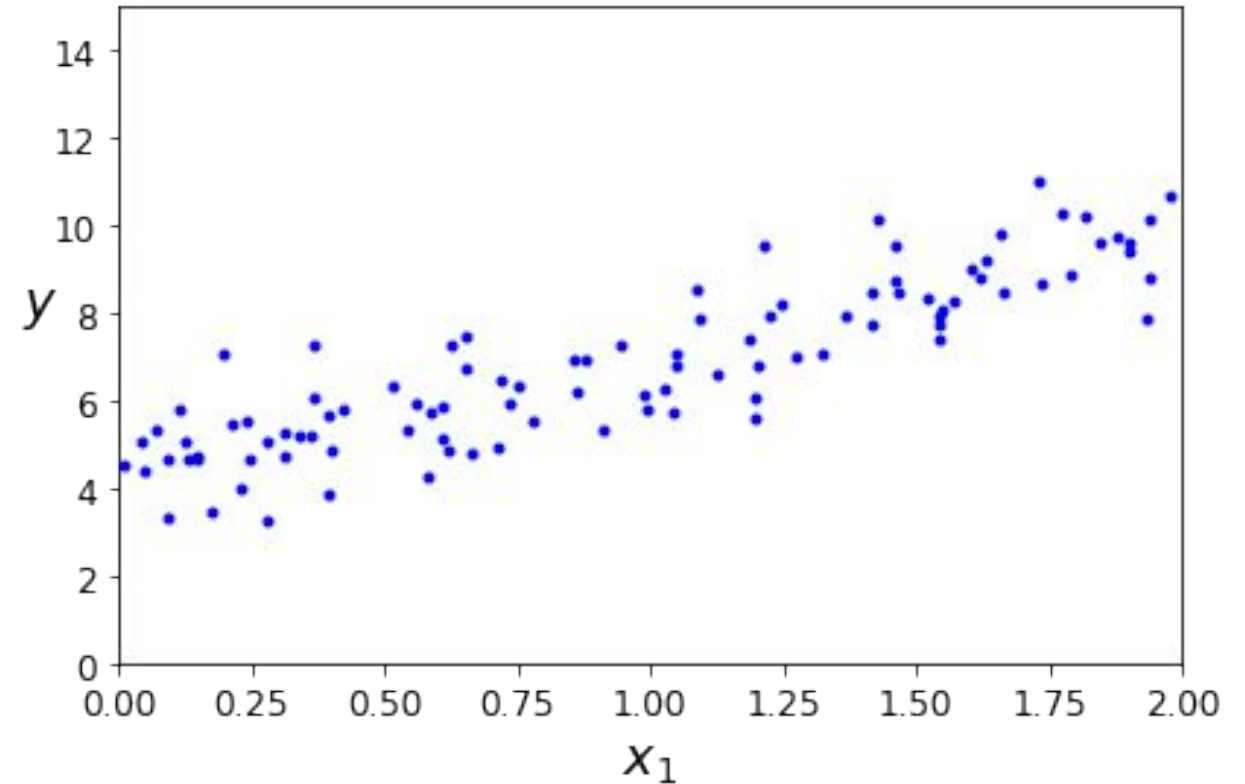
Regression

- Relationship between one dependent variable and a series of other variables (independent variables)



Regression

- Two very different ways to train it:
 - Using a direct “closed-form” equation that directly computes the model parameters that best fit the model to the training set (i.e., the model parameters that minimize the cost function over the training set).
 - Using an iterative optimization approach, called Gradient Descent (GD), that gradually tweaks the model parameters to minimize the cost function over the training set, eventually converging to the same set of parameters as the first method.



Regression

- Linear Regression
- Polynomial Regression
- Logistic Regression
- Softmax Regression

Linear Regression

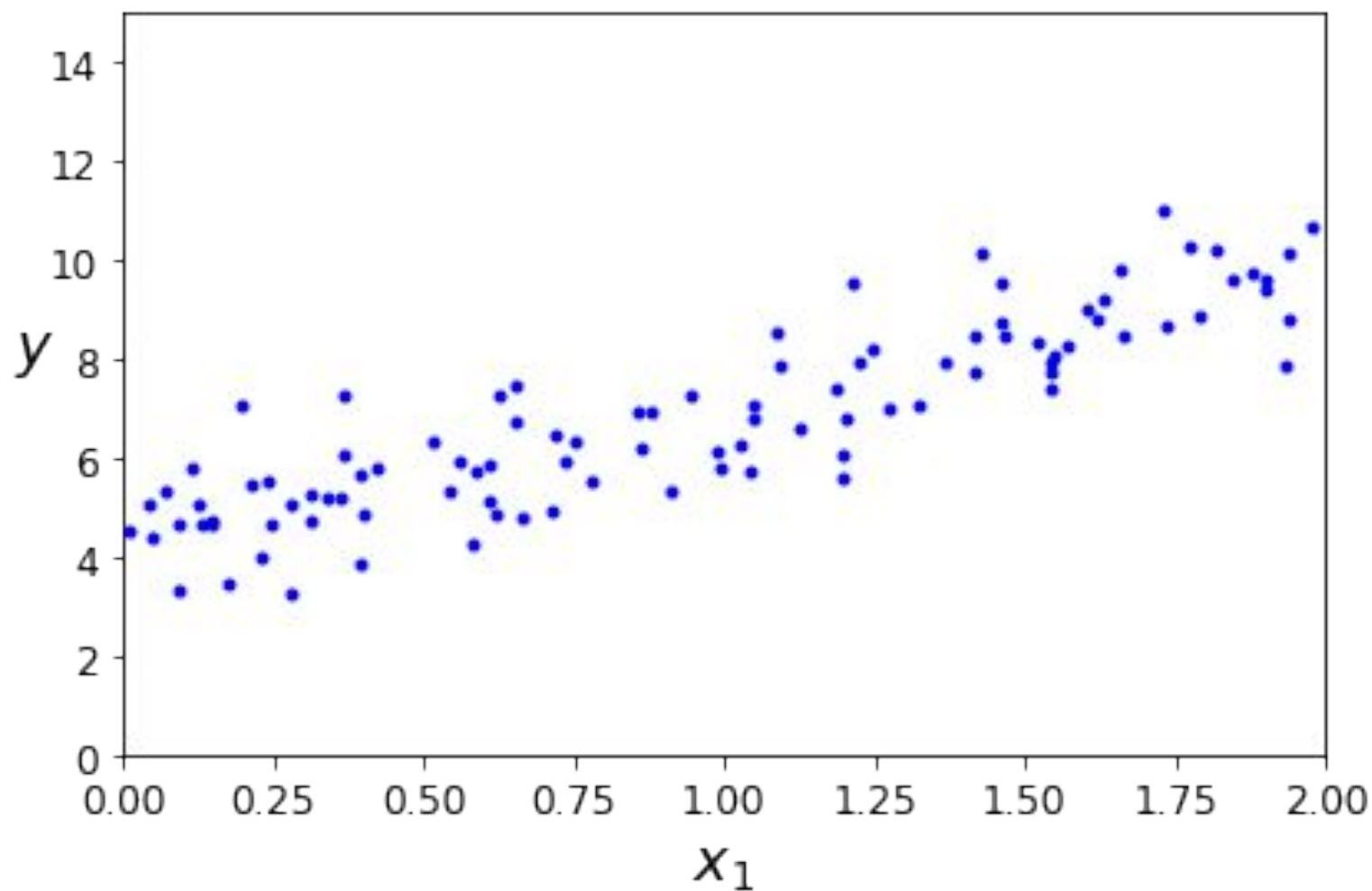
- a linear model makes a prediction by simply computing a weighted sum of the input features, plus a constant called the bias term (also called the intercept term)

Equation 4-1. Linear Regression model prediction

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Equation 4-3. MSE cost function for a Linear Regression model

$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right)^2$$



Normal Equation

$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right)^2$$

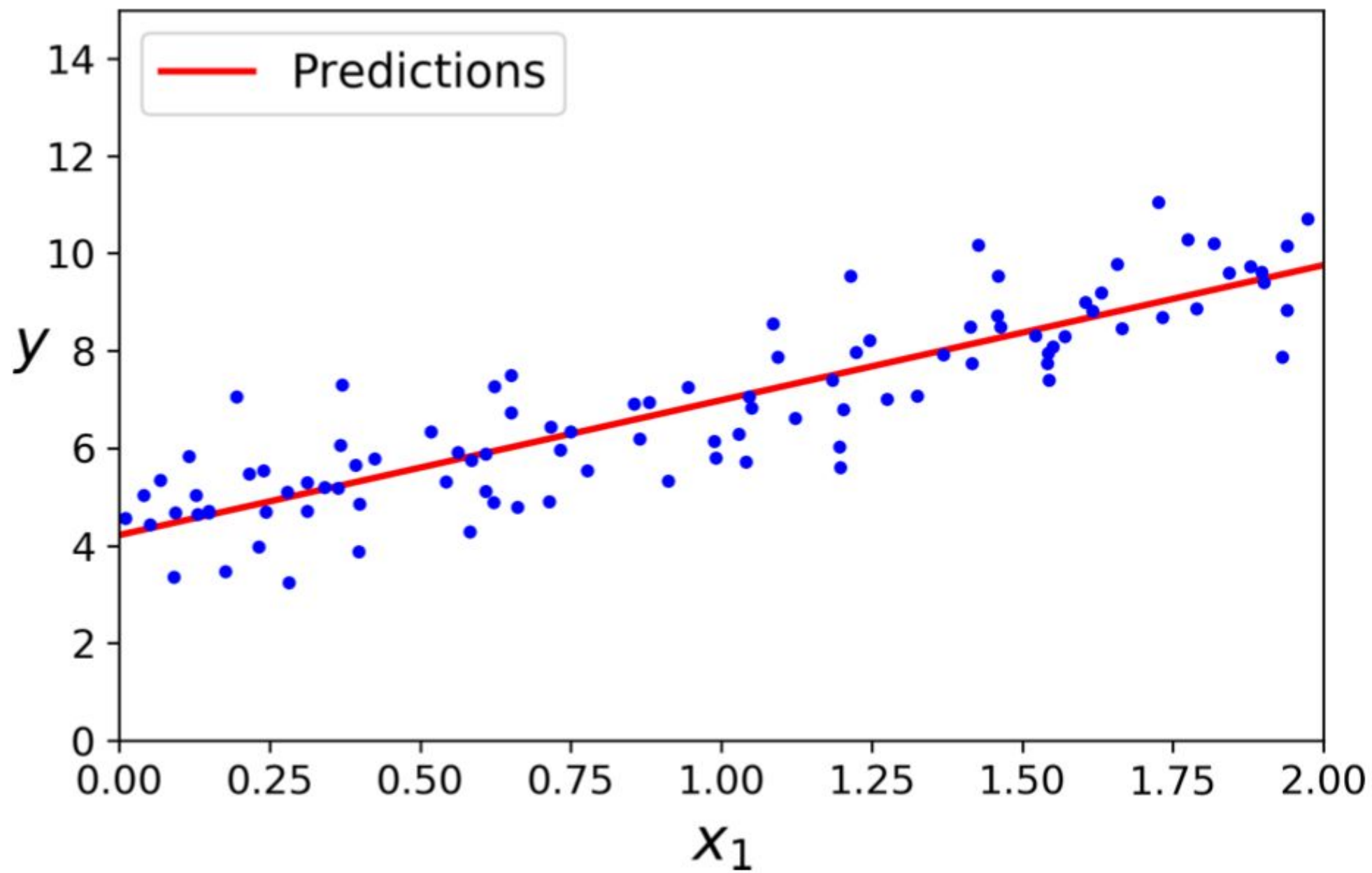
- To find the value of θ that minimizes the cost function, there is a closed-form solution—in other words, a mathematical equation that gives the result directly. This is called the Normal Equation

Equation 4-4. Normal Equation

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

```
theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)
```

```
[4.21509616],[2.77011339]
```

Linear Regression

- Performing linear regression using Scikit-Learn is quite simple:

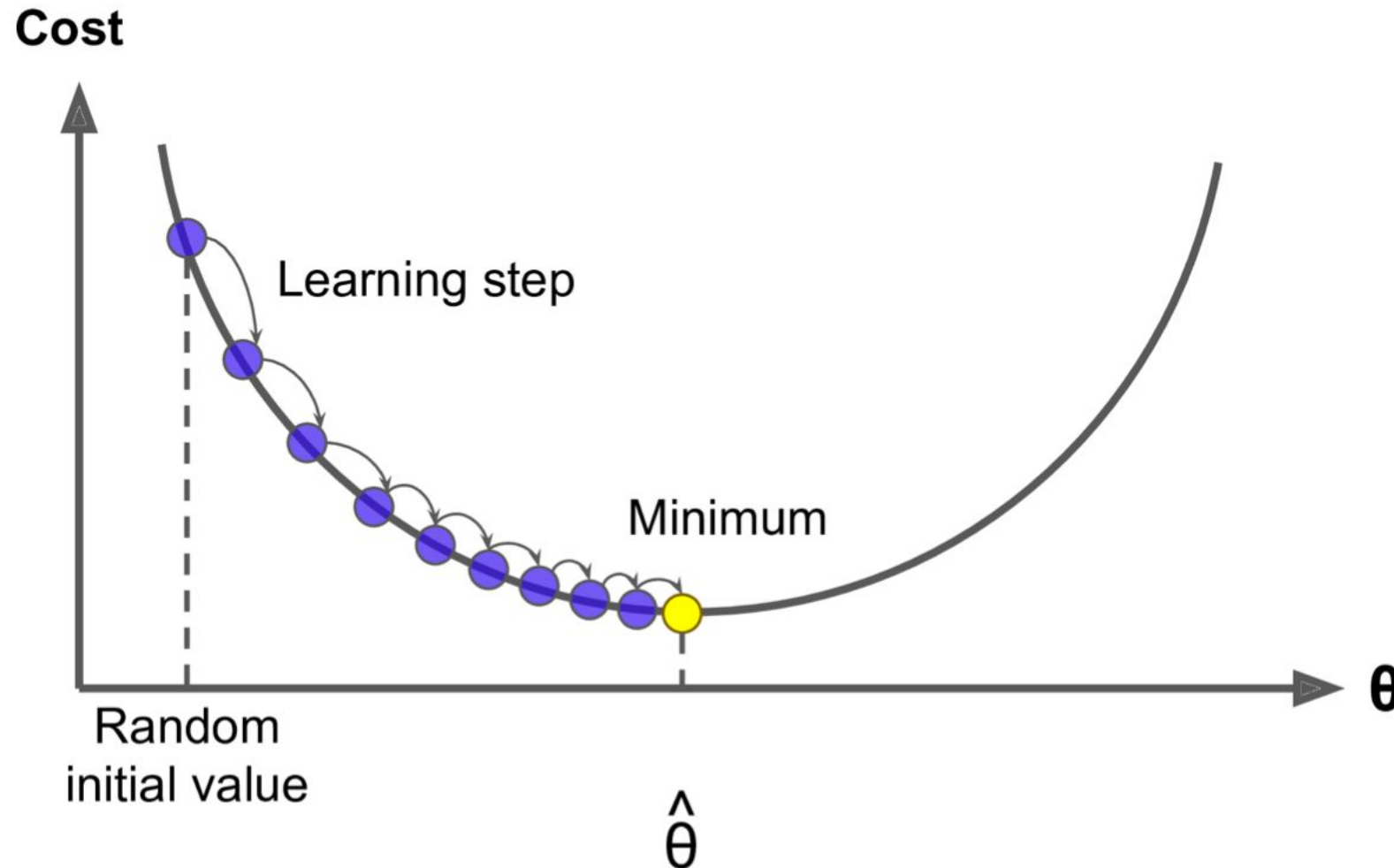
```
>>> from sklearn.linear_model import LinearRegression
>>> lin_reg = LinearRegression()
>>> lin_reg.fit(X, y)
>>> lin_reg.intercept_, lin_reg.coef_ (array([4.21509616]),
array([[2.77011339]]))
>>> lin_reg.predict(X_new)
array([[4.21509616], [9.75532293]])
```

- High complexity – increasing number of features

Gradient Descent

$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2$$

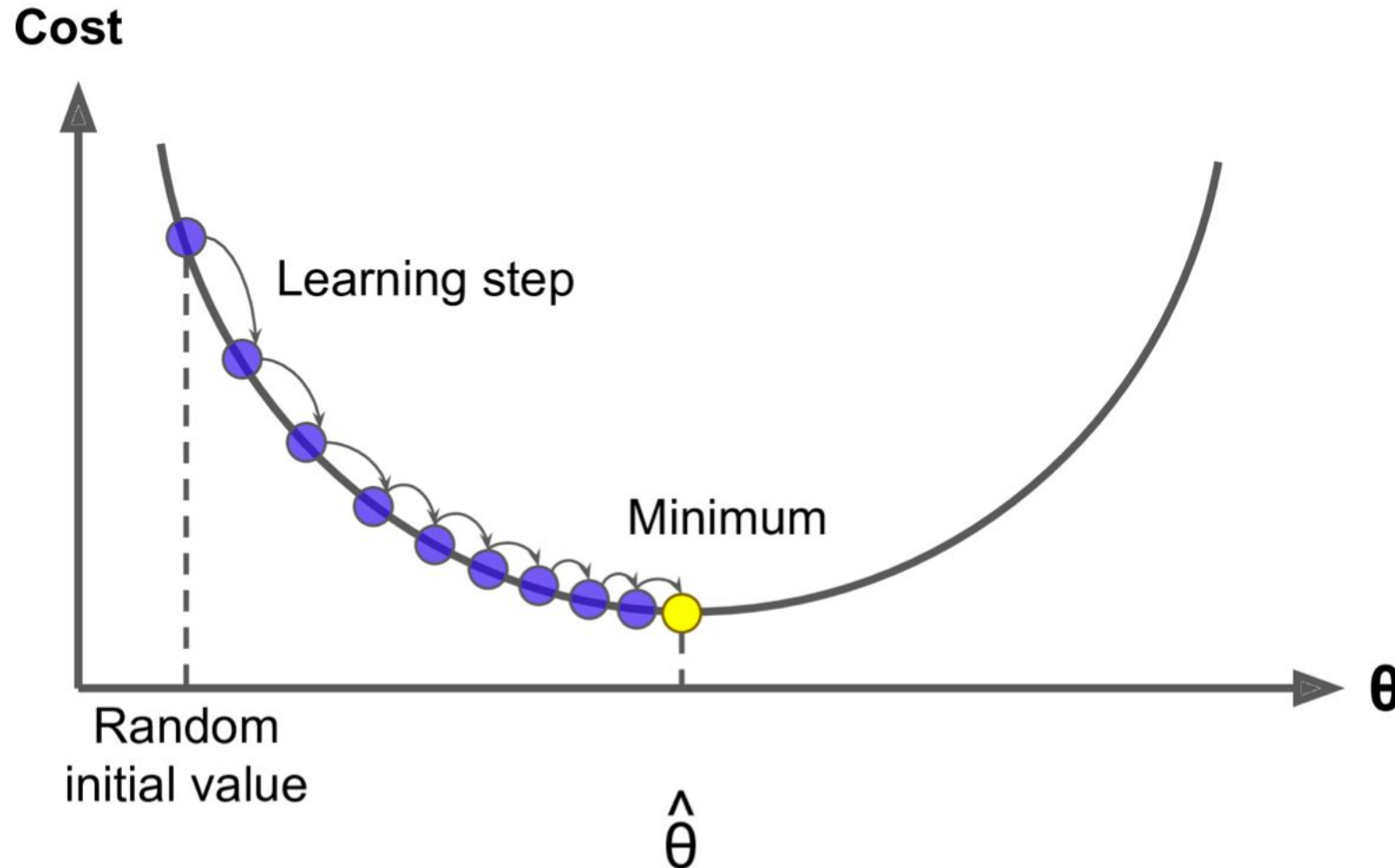
- Gradient Descent is a very generic optimization algorithm capable of finding optimal solutions to a wide range of problems. The general idea of Gradient Descent is to **tweak** parameters iteratively in order to **minimise a cost function**.



Gradient Descent

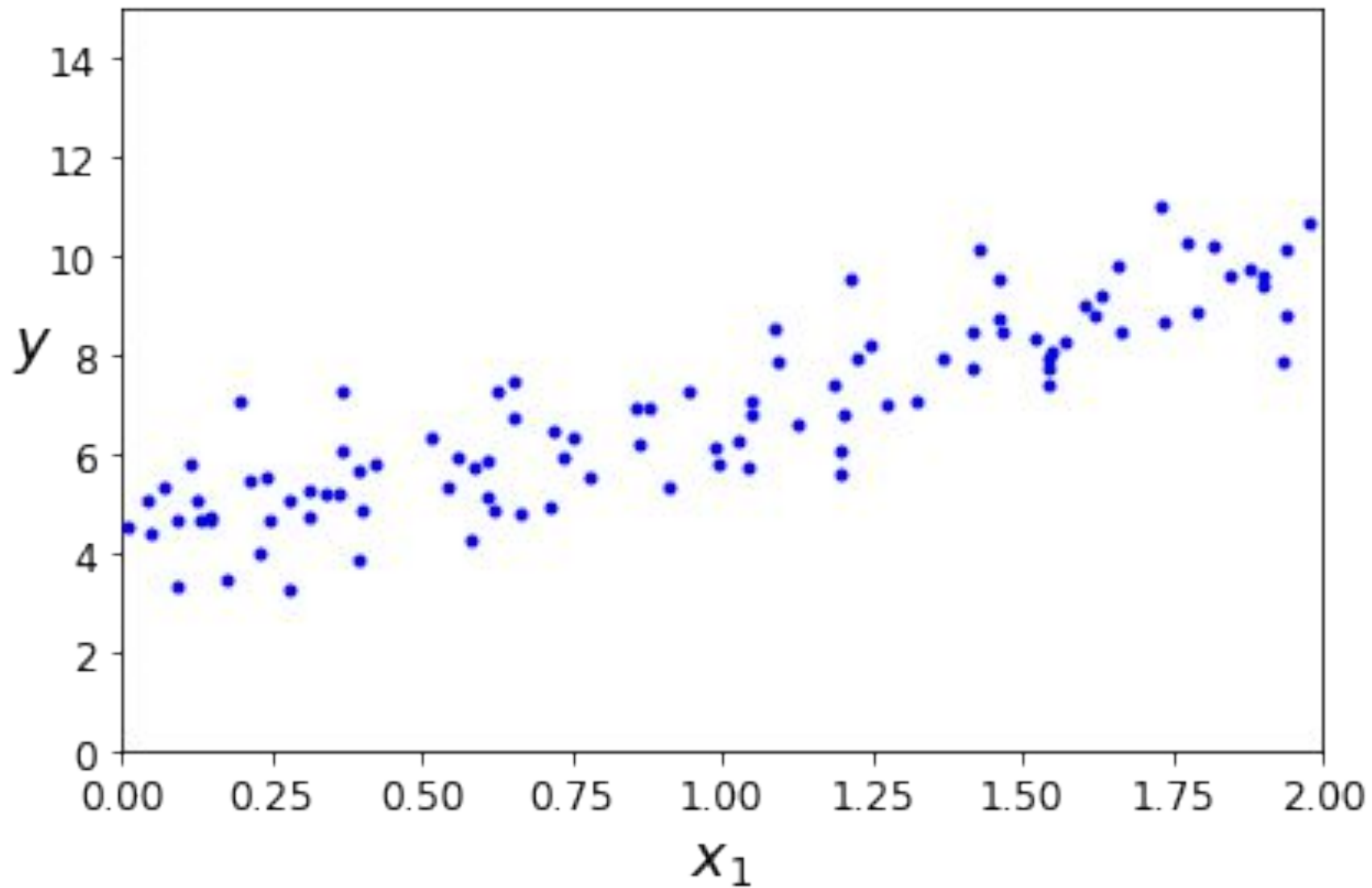
$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m (\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)})^2$$

- Concretely, you start by filling $\boldsymbol{\theta}$ with random values (this is called random initialization), and then you improve it gradually, taking one baby step at a time, each step attempting to decrease the cost function (e.g., the MSE), until the algorithm converges to a minimum



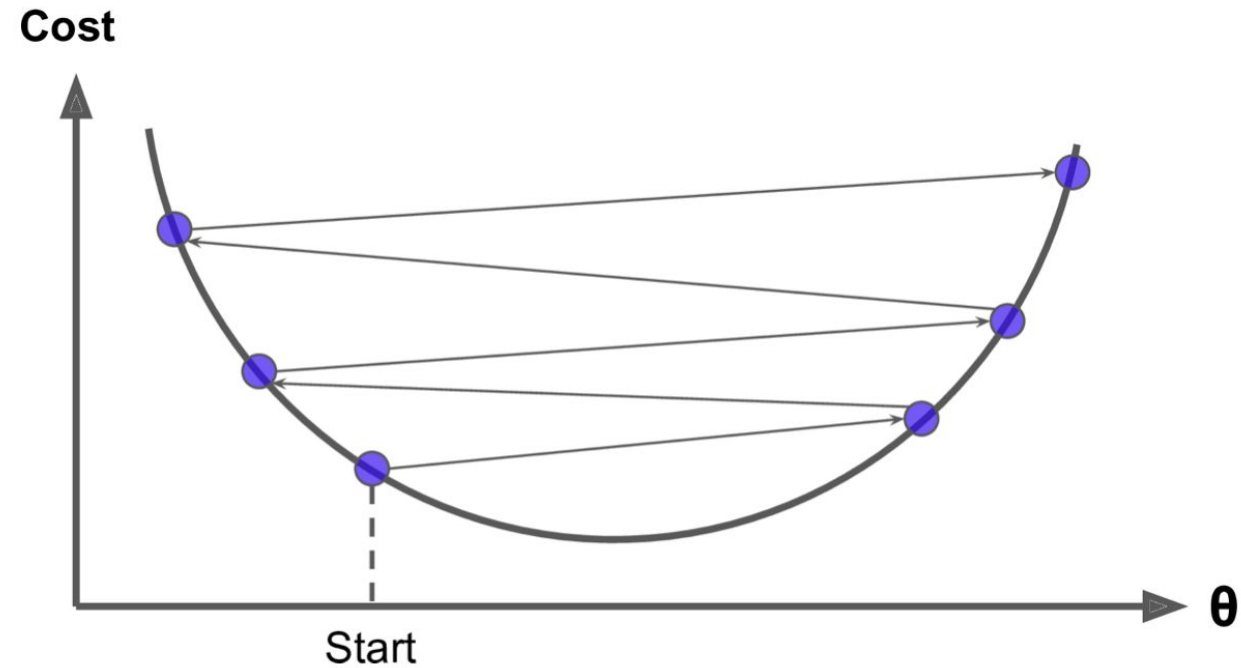
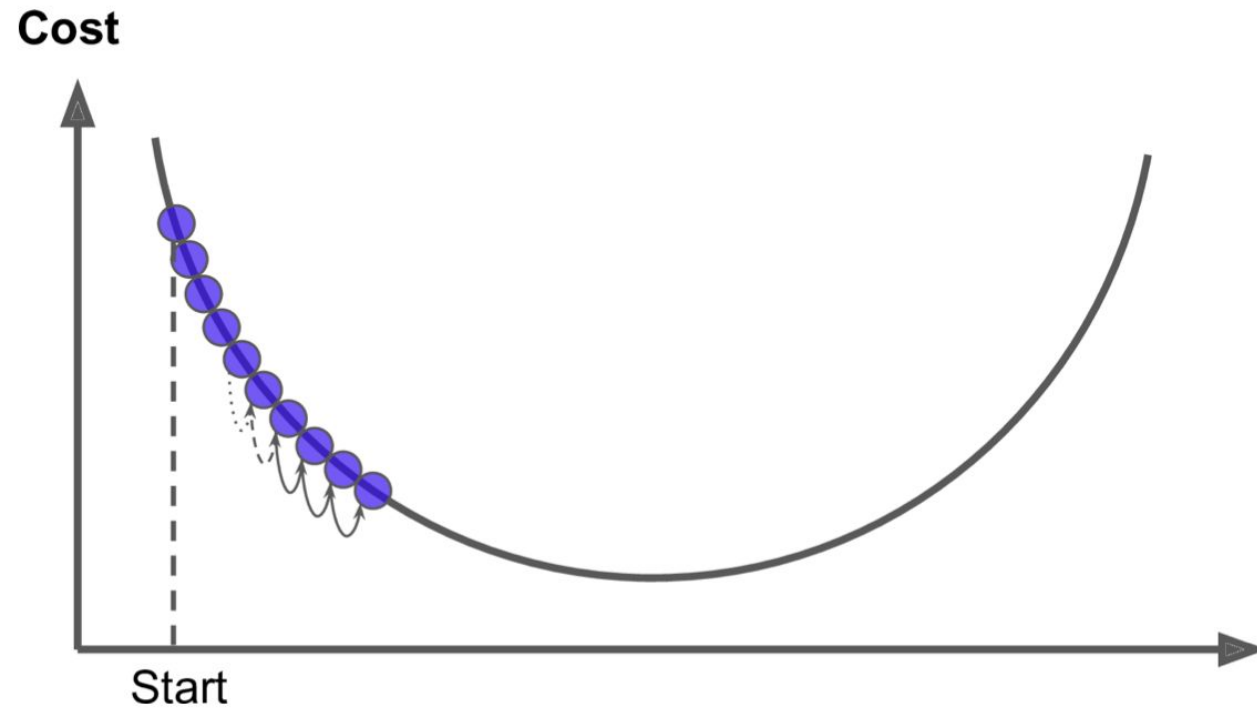
$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

$$\text{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^m \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right)^2$$



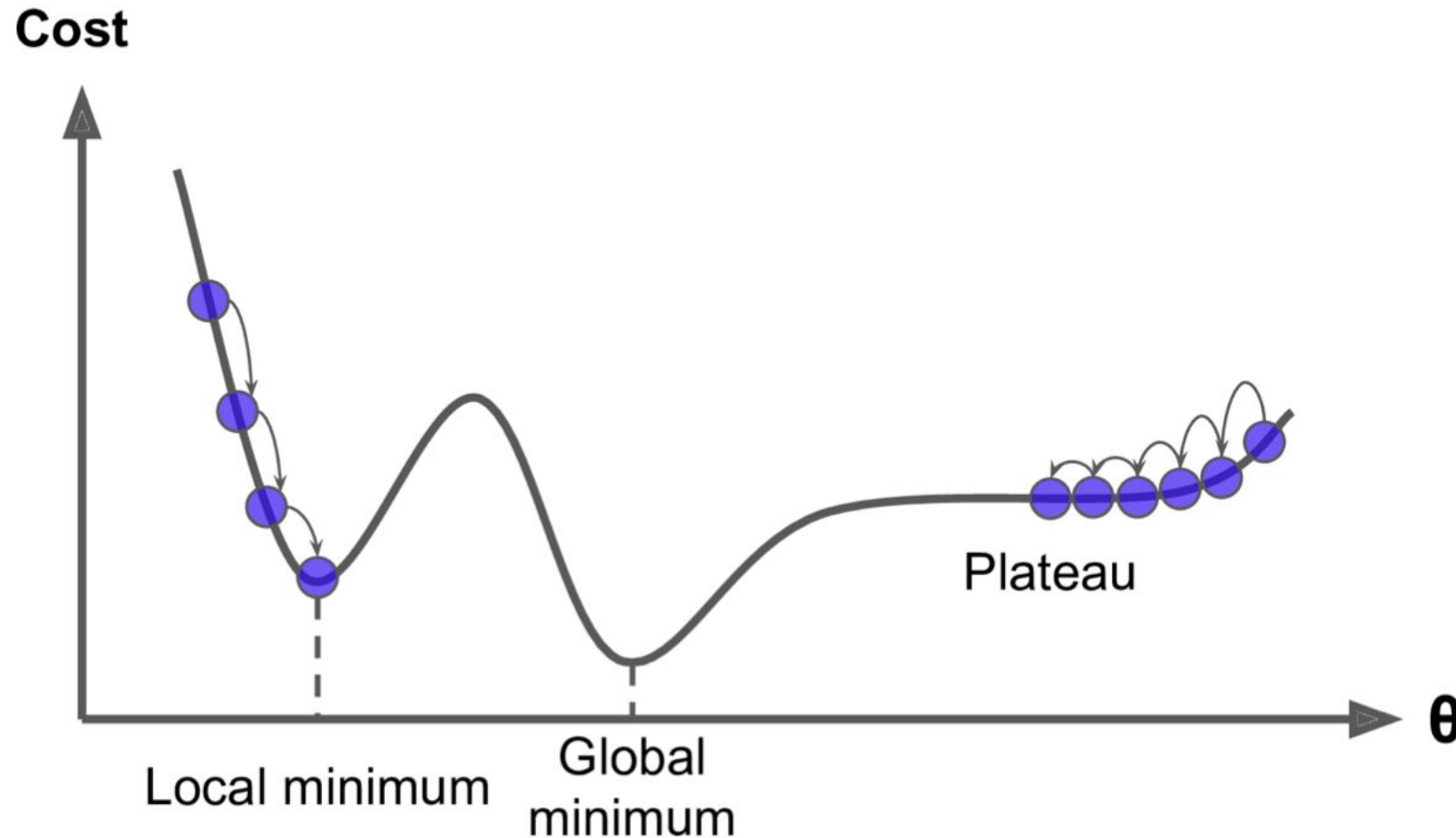
Gradient Descent

- An important parameter in Gradient Descent is the size of the steps, determined by the learning rate hyperparameter.



Gradient Descent

- two main challenges with Gradient Descent: if the random initialization starts the algorithm on the left, then it will converge to a **local minimum**, which is not as good as the global minimum. If it starts on the right, then it will take a very long time to cross the **plateau**, and if you stop too early you will never reach the global minimum.



Batch Gradient Descent

- To implement Gradient Descent, you need to compute the gradient of the cost function with regards to each model parameter θ_j . multiply the gradient vector by η to determine the size of the downhill step.

Equation 4-6. Gradient vector of the cost function

$$\nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta}) = \begin{pmatrix} \frac{\partial}{\partial \theta_0} \text{MSE}(\boldsymbol{\theta}) \\ \frac{\partial}{\partial \theta_1} \text{MSE}(\boldsymbol{\theta}) \\ \vdots \\ \frac{\partial}{\partial \theta_n} \text{MSE}(\boldsymbol{\theta}) \end{pmatrix} = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\boldsymbol{\theta} - \mathbf{y})$$

Equation 4-7. Gradient Descent step

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \text{MSE}(\boldsymbol{\theta})$$

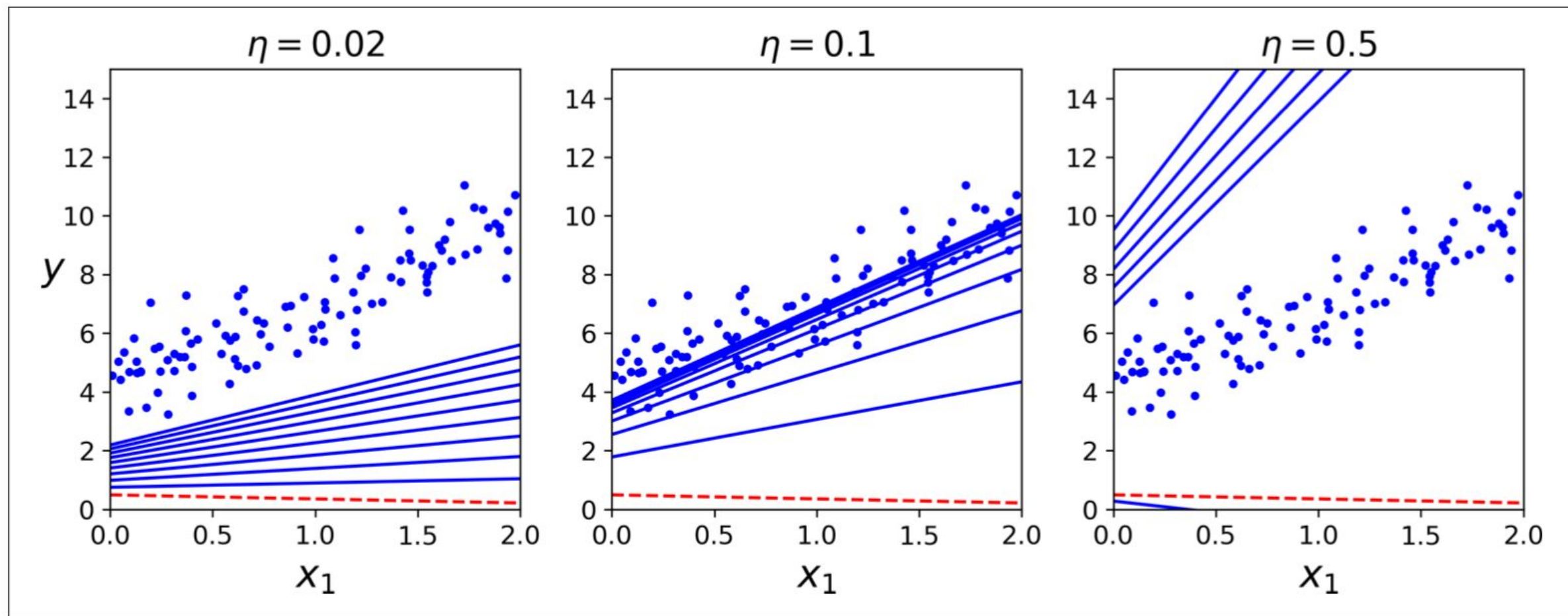
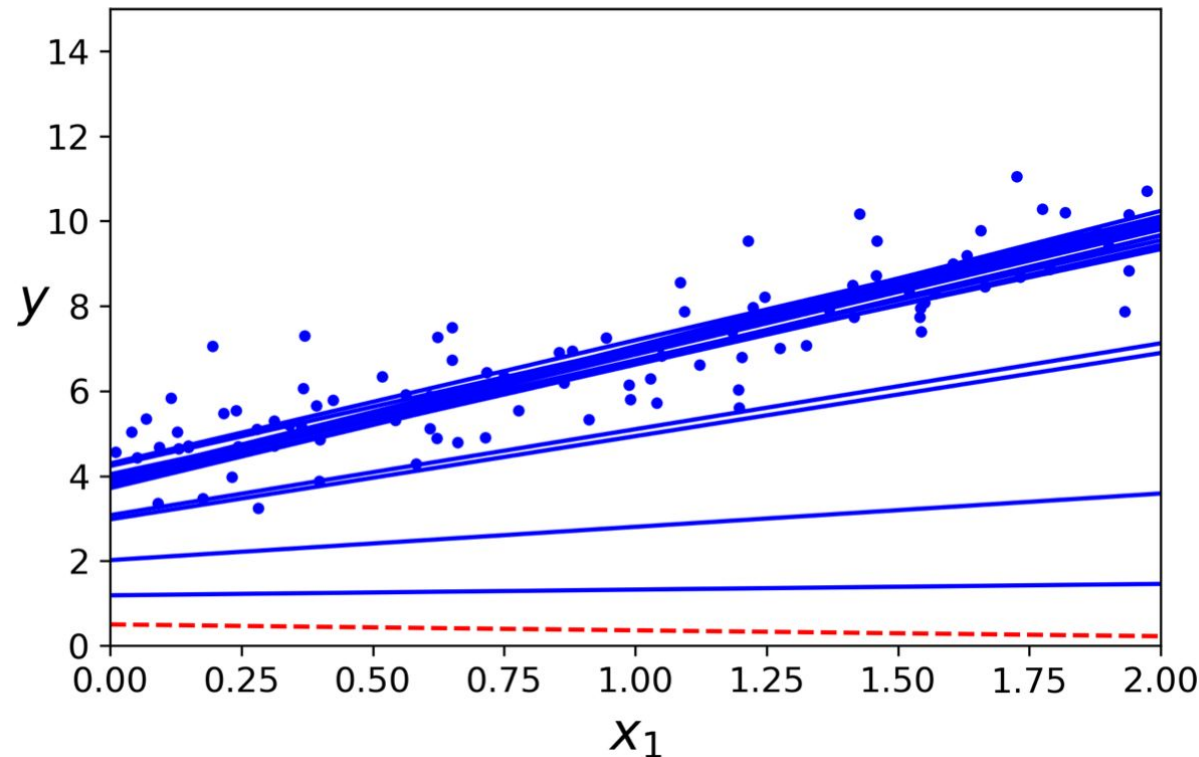


Figure 4-8. Gradient Descent with various learning rates

Stochastic Gradient Descent

- The main problem with Gradient Descent is the fact that it uses the **whole training set** to compute the gradients at every step, which makes it **very slow** when the training set is large.
- At the opposite extreme, **Stochastic Gradient Descent** just picks a **random instance** in the training set at every step and computes the gradients based only on that single instance. Obviously this makes the algorithm much faster since it has very little data to manipulate at every iteration.



Stochastic Gradient Descent

- On the other hand, due to its stochastic (i.e., random) nature, this algorithm is much less regular than Batch Gradient Descent: instead of gently decreasing until it reaches the minimum, the cost function will bounce up and down, decreasing only on average. Over time it will end up very close to the minimum, but once it gets there it will continue to bounce around, never settling down.
- One solution to this dilemma is to gradually reduce the learning rate. The steps start out large (which helps make quick progress and escape local minima), then get smaller and smaller, allowing the algorithm to settle at the global minimum.

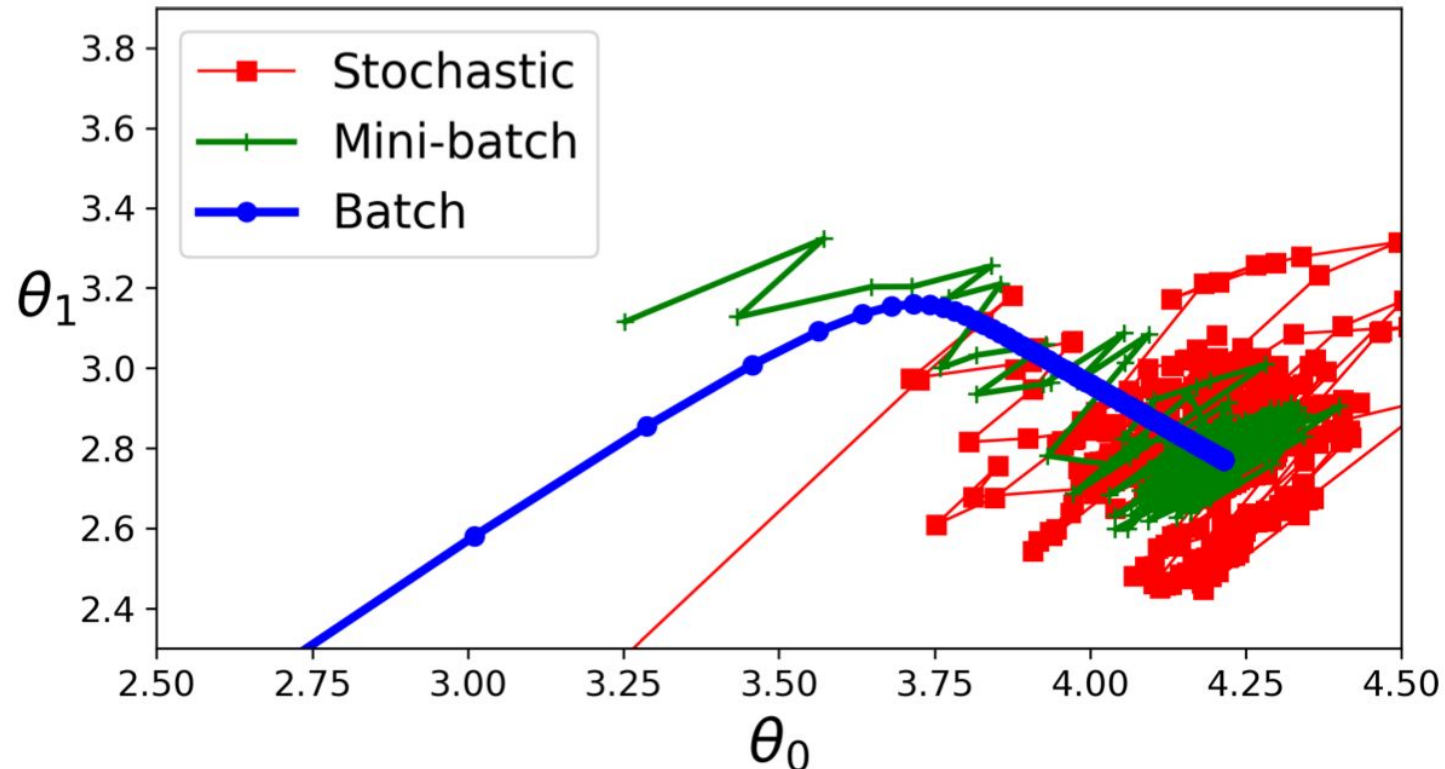
Stochastic Gradient Descent

```
from sklearn.linear_model import SGDRegressor  
sgd_reg = SGDRegressor(max_iter=1000, tol=1e-3, penalty=None, eta0=0.1)  
sgd_reg.fit(X, y.ravel())
```

- To perform Linear Regression using SGD with Scikit-Learn, you can use the *SGDRegressor* class, which defaults to optimizing the squared error cost function.
- The above code runs for maximum 1000 epochs (`max_iter=1000`) or until the loss drops by less than $1e-3$ during one epoch (`tol=1e-3`), starting with a learning rate of 0.1 (`eta0=0.1`), using the default learning schedule (different from the preceding one), and it does not use any regularization (`penalty=None`):

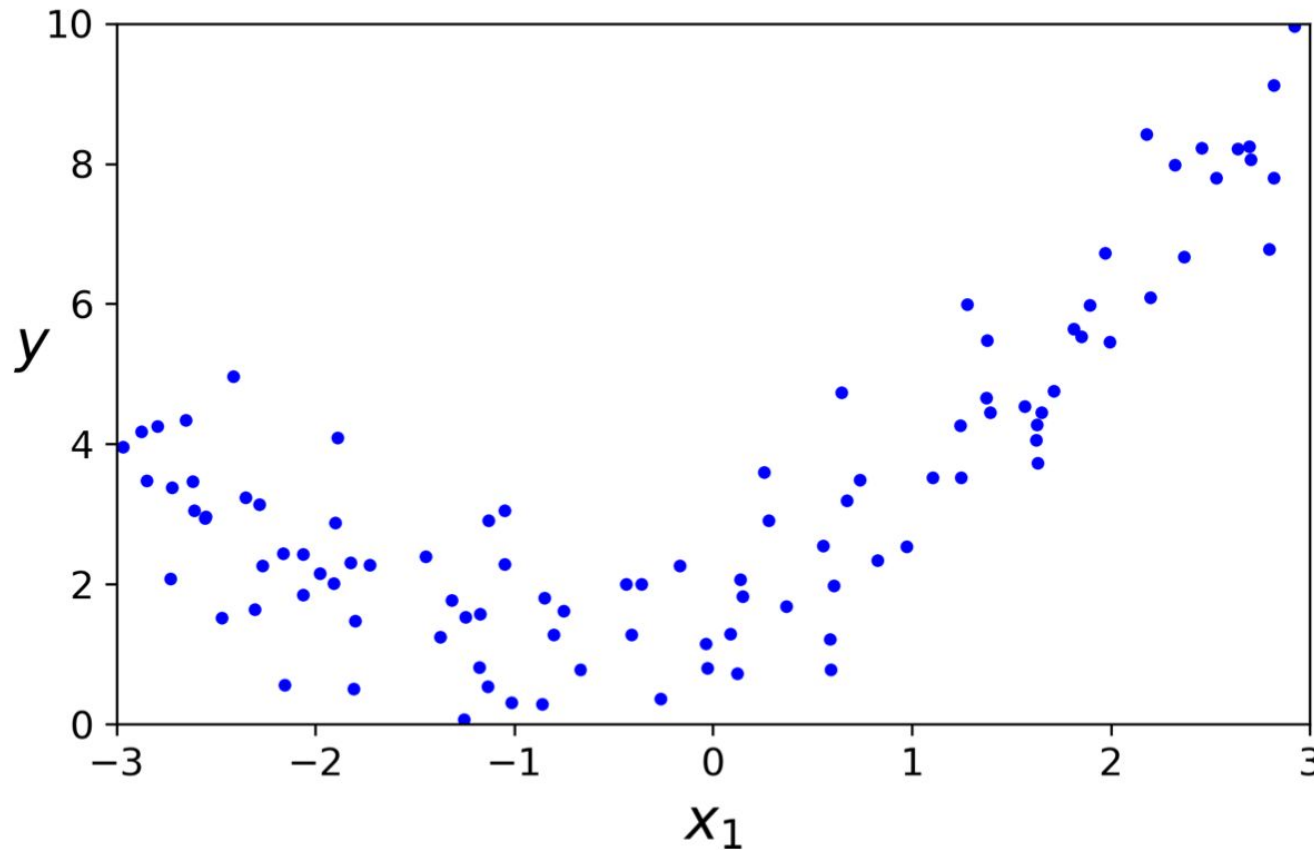
Mini-batch Gradient Descent

- instead of computing the gradients based on the full training set (as in Batch GD) or based on just one instance (as in Stochastic GD), Mini-batch GD computes the gradients on **small random sets** of instances called **mini-batches**.
- The main advantage of Mini-batch GD over Stochastic GD is that you can get a performance boost from hardware optimization of matrix operations, especially when using GPUs.



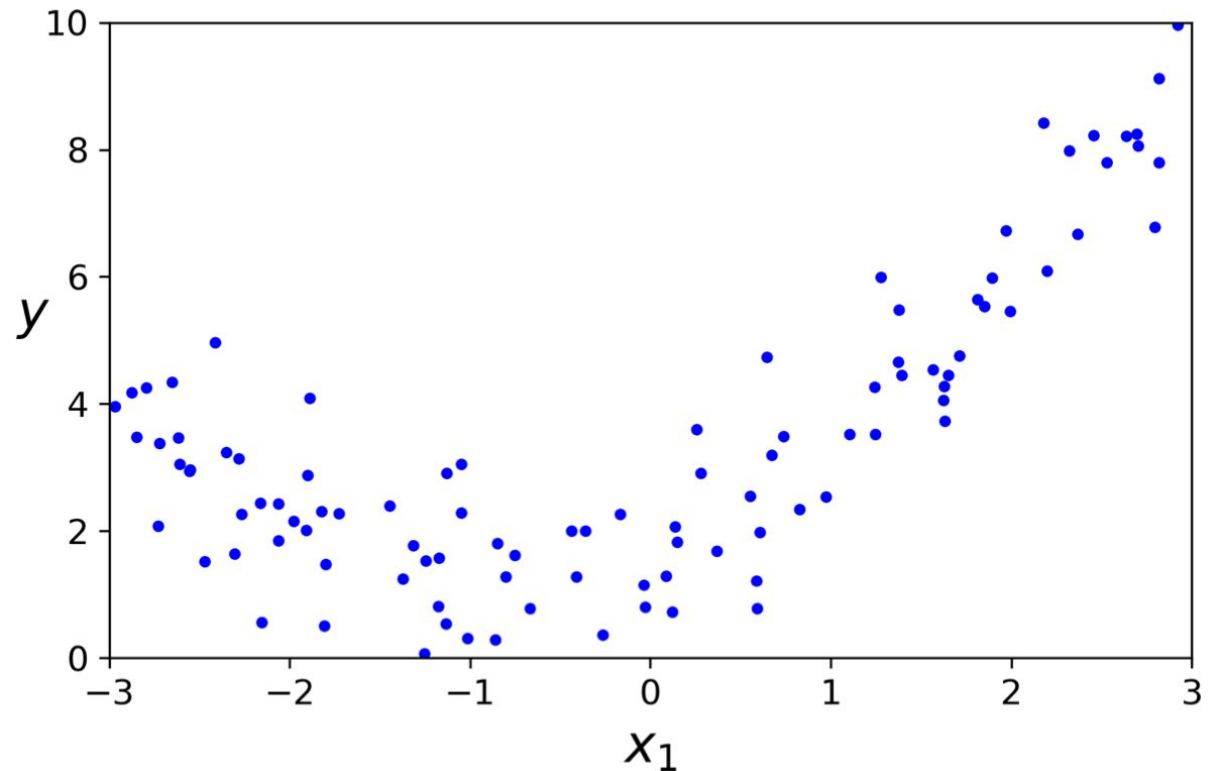
Polynomial Regression

- Clearly, a straight line will never fit this data properly. So let's use Scikit-Learn's PolynomialFeatures class to transform our training data, adding the square (2nd-degree polynomial) of each feature in the training set as new features.



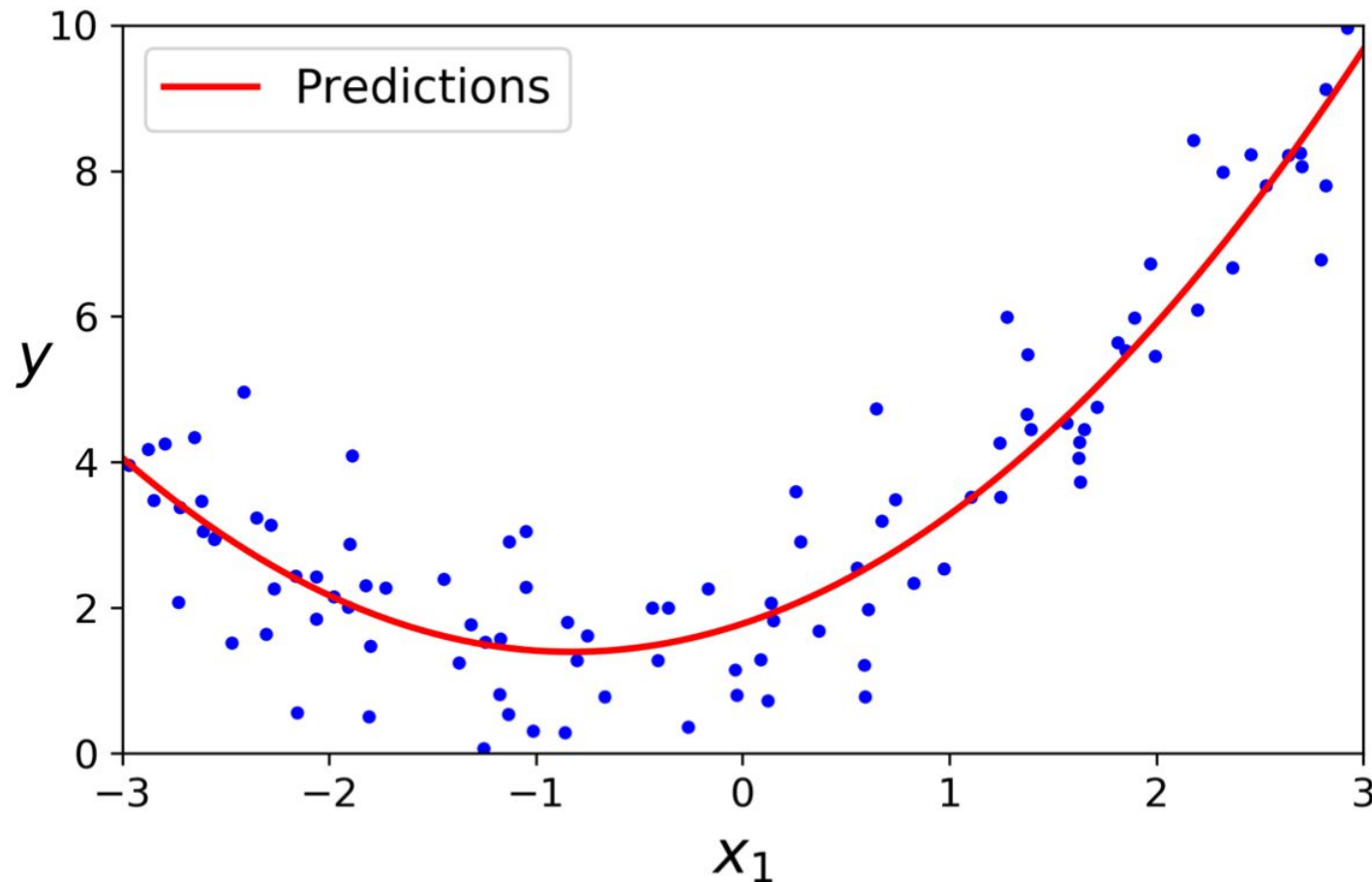
Polynomial Regression

```
>>> from sklearn.preprocessing import PolynomialFeatures
>>> poly_features = PolynomialFeatures(degree=2, include_bias=False)
>>> X_poly = poly_features.fit_transform(X)
>>> X[0]
array([-0.75275929])
>>> X_poly[0]
array([-0.75275929, 0.56664654])
```



Polynomial Regression

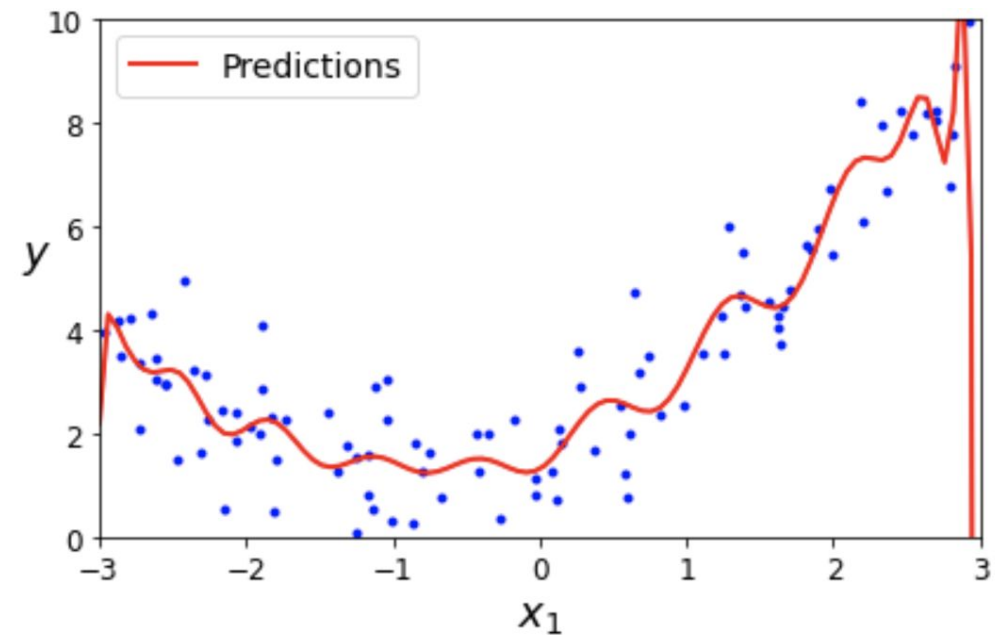
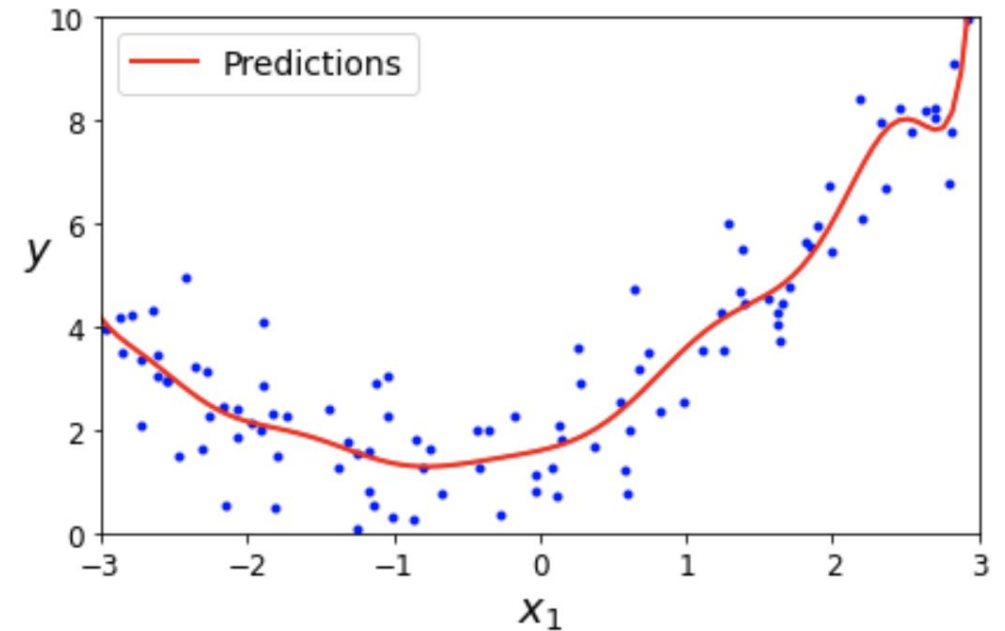
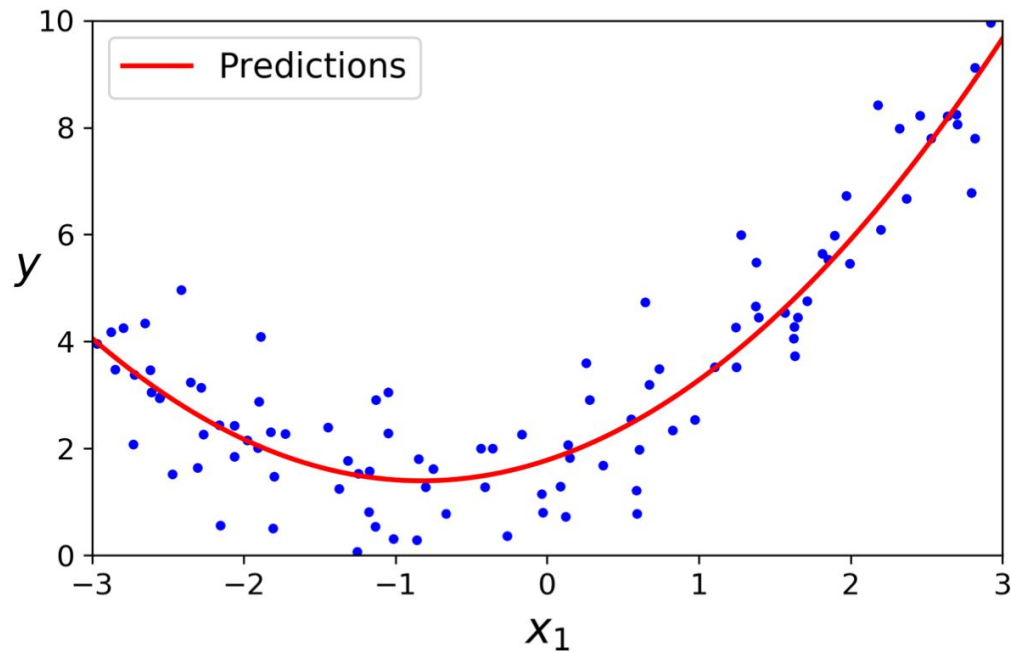
```
>>> lin_reg = LinearRegression()  
>>> lin_reg.fit(X_poly, y)  
>>> lin_reg.intercept_, lin_reg.coef_  
(array([1.78134581]), array([[0.93366893, 0.56456263]]))
```

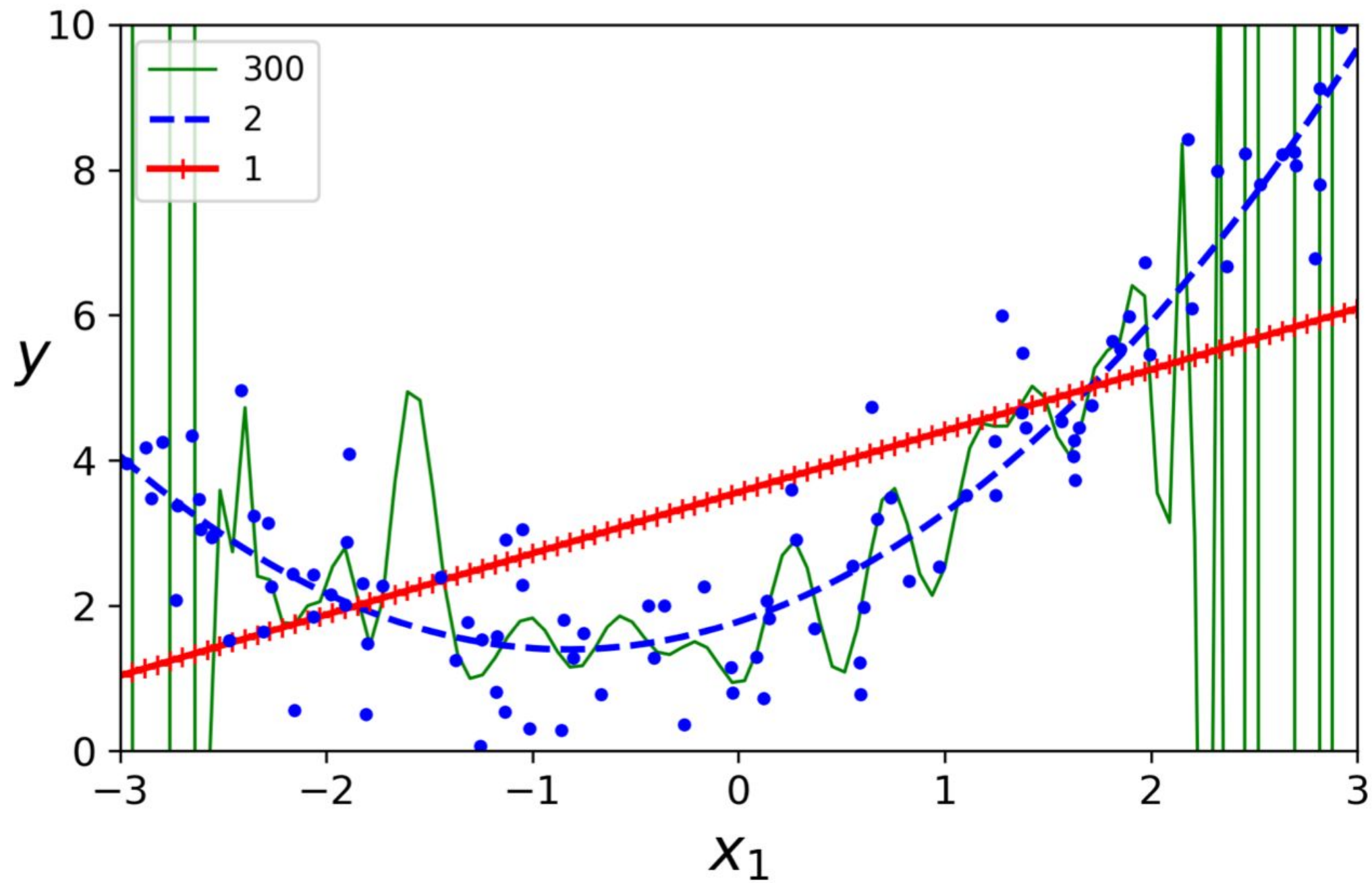


$$\hat{y} = 0.56x_1^2 + 0.93x_1 + 1.78$$

Polynomial Regression

- If you perform high-degree Polynomial Regression, you will likely fit the training data much better than with plain Linear Regression.





Further reading - Regularization

- a good way to reduce overfitting is to regularize the model (i.e., to constrain it): the fewer degrees of freedom it has, the harder it will be for it to overfit the data.
- A regularization term is added to the cost function. This forces the learning algorithm to not only fit the data but also keep the model weights as small as possible

Equation 4-8. Ridge Regression cost function

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \sum_{i=1}^n \theta_i^2$$

Equation 4-10. Lasso Regression cost function

$$J(\boldsymbol{\theta}) = \text{MSE}(\boldsymbol{\theta}) + \alpha \sum_{i=1}^n |\theta_i|$$